

# BOOK DOCTOR MANAGEMENT SYSTEM

A Full-Stack Healthcare Appointment Management Platform

---

|                         |                                                                                           |
|-------------------------|-------------------------------------------------------------------------------------------|
| <b>Technology Stack</b> | MERN Stack (MongoDB, Express.js, React.js, Node.js)                                       |
| <b>Architecture</b>     | MVC (Model View Controller)                                                               |
| <b>Authentication</b>   | JWT + bcrypt                                                                              |
| <b>Repository</b>       | <a href="https://github.com/GOWD-HUB/book-a-doctor">github.com/GOWD-HUB/book-a-doctor</a> |

PROJECT DONE BY  
**R. THIMME GOWD & TEAM**  
Yringam Rajesh • Karanam Sreenivasulu

# TABLE OF CONTENTS

Chapter 1 — Introduction

---

Chapter 2 — Problem Statement

---

Chapter 3 — Existing System

---

Chapter 4 — Proposed System

---

Chapter 5 — Objectives of the Project

---

Chapter 6 — Scope of the Project

---

Chapter 7 — Technologies Used

---

Chapter 8 — System Architecture

---

Chapter 9 — Technical Architecture

---

Chapter 10 — Database Design

---

Chapter 11 — Frontend Development

---

Chapter 12 — Backend Development

---

Chapter 13 — API Documentation

---

Chapter 14 — Project Execution

---

Chapter 15 — Testing

---

Chapter 16 — Project Results

---

Chapter 17 — Future Enhancements

---

Chapter 18 — Conclusion

---

---

References

---

GitHub Repository

---

Appendix

---

Document Summary

# ACKNOWLEDGEMENT

We would like to express our sincere gratitude to everyone who supported and guided us throughout the development of the Book Doctor Management System. This project reflects our collective effort in applying the MERN stack to build a secure and practical healthcare appointment platform.

## Project Team

- R. Thimme Gowd — Team Lead
- Yringam Rajesh
- Karanam Sreenivasulu

# INTRODUCTION

Healthcare accessibility and efficient appointment management remain persistent challenges for patients and healthcare providers alike. Traditional appointment booking methods rely heavily on phone calls, walk-in registrations, and manual record keeping, which are time-consuming, error-prone, and difficult to scale.

The Book Doctor Management System is a full-stack web application built on the MERN stack (MongoDB, Express.js, React.js, Node.js) that digitizes the entire appointment booking workflow. It connects patients, doctors, and administrators on a single secure platform, allowing patients to search for doctors, book appointments online, and track their appointment history, while doctors and administrators manage requests and records through dedicated dashboards.

This report documents the design, development, and implementation of the system, covering its objectives, scope, technologies, architecture, database design, frontend and backend development, testing, and results.

## CHAPTER 2

# PROBLEM STATEMENT

Booking a doctor's appointment through conventional means presents several practical difficulties for both patients and healthcare providers.

### Key Problems Identified

- Patients often need to call clinics or visit in person to check doctor availability, which is inconvenient and time-consuming.
- Manual appointment registers are prone to double-booking, lost records, and scheduling conflicts.
- Doctors have limited visibility into their upcoming appointments and cannot easily approve, reject, or reschedule requests.
- Administrators lack a centralized system to monitor doctors, patients, and appointment activity.
- There is no secure digital record of patient appointment history for future reference.
- Paper-based or spreadsheet-based systems do not scale well as the number of patients and doctors grows.

These challenges highlight the need for a centralized, secure, and automated appointment management platform that streamlines communication between patients, doctors, and administrators.

# EXISTING SYSTEM

Most small and mid-sized healthcare providers currently rely on manual or semi-digital methods to manage patient appointments.

### Characteristics of the Existing System

- Appointments are scheduled via phone calls or in-person visits.
- Records are maintained in physical registers or basic spreadsheets.
- There is no real-time visibility of doctor availability for patients.
- Appointment confirmations, cancellations, and updates are communicated manually.
- No centralized dashboard exists for administrators to track overall activity.
- Patient history is difficult to retrieve quickly, and there is a higher risk of data loss.

### Limitations

- Time-consuming and inconvenient for patients.
- High chance of human error in scheduling.
- No role-based access or security for sensitive patient data.
- Poor scalability as patient and doctor volume increases.
- Lack of analytics or reporting for administrators.

# PROPOSED SYSTEM

The proposed Book Doctor Management System addresses the limitations of the existing system by providing a fully digital, secure, and role-based web application for healthcare appointment management.

### Key Features of the Proposed System

- Online appointment booking available to patients at any time, without phone calls or visits.
- Real-time doctor availability and profile information accessible to patients.
- Secure authentication using JWT and bcrypt password hashing to protect user data.
- Role-based dashboards for Administrators, Doctors, and Patients with distinct functionalities.
- Structured appointment workflow supporting creation, confirmation, cancellation, and completion.
- Centralized MongoDB database ensuring consistent and reliable data storage.
- RESTful API architecture enabling smooth communication between the React frontend and Node.js backend.
- Dashboard analytics for administrators to monitor users, doctors, patients, and appointments in real time.

### Advantages Over the Existing System

- Reduces manual effort and eliminates scheduling conflicts.
- Improves accessibility — patients can book appointments anytime, anywhere.
- Provides a secure, auditable digital record of all appointments.
- Scales easily to support a growing number of doctors and patients.
- Offers a modern, responsive user interface across devices.

## CHAPTER 5

# OBJECTIVES OF THE PROJECT

The primary objective of the Book Doctor Management System is to provide an efficient, secure, and user-friendly web application for managing doctor appointments. The project aims to simplify the interaction between patients, doctors, and administrators while reducing manual effort in healthcare appointment management.

### Main Objectives

#### 1. Digital Appointment Booking

Provide an online platform where patients can easily book appointments with available doctors.

#### 2. Secure User Authentication

Implement secure registration and login using JWT (JSON Web Token) and bcrypt password hashing.

#### 3. Role-Based Access Control

Provide different functionalities for Administrator, Doctor, and Patient roles.

#### 4. Doctor Management

Allow administrators to manage doctor profiles, update information, and monitor healthcare providers.

#### 5. Patient Management

Maintain patient information securely and enable profile updates.

#### 6. Appointment Management

Allow appointment creation, confirmation, cancellation, and completion through an organized workflow.

#### 7. Dashboard Analytics

Provide real-time dashboard statistics for administrators regarding users, doctors, patients, and appointments.

#### 8. Responsive User Interface

Develop an intuitive and responsive frontend using React.js and CSS.

## 9. Database Management

Store and manage application data efficiently using MongoDB and Mongoose.

## 10. REST API Integration

Develop secure RESTful APIs for seamless communication between frontend and backend.

## CHAPTER 6

# SCOPE OF THE PROJECT

The Book Doctor Management System is designed to automate healthcare appointment scheduling and improve communication between patients and healthcare providers.

### Scope Includes

#### Patient Module

- Patient Registration
- Secure Login
- View Available Doctors
- Book Appointments
- View Appointment History
- Update Profile

#### Doctor Module

- Doctor Login
- View Assigned Appointments
- Approve Appointment Requests
- Reject Appointment Requests
- Complete Appointments
- Manage Profile

#### Administrator Module

- Dashboard Analytics
- View All Doctors
- View All Patients
- View All Appointments
- Manage System Records

#### Backend Scope

- JWT Authentication
- Password Encryption

- RESTful API Development
- MongoDB Database Integration
- MVC Architecture
- Error Handling

### Future Scope

The system can be extended by integrating:

- Online Payment Gateway
- Video Consultation
- AI Chatbot
- Email Notifications
- SMS Notifications
- Electronic Prescription
- Medical Report Upload
- Doctor Ratings
- Mobile Application

## CHAPTER 7

# TECHNOLOGIES USED

### React.js

React.js is an open-source JavaScript library used for developing dynamic user interfaces. It enables component-based development and provides efficient rendering through the Virtual DOM.

#### Advantages

- Component Reusability
- Fast Rendering
- Virtual DOM
- Easy State Management
- High Performance

#### Usage in Project

- Home Page
- Login
- Registration
- Dashboard
- Appointment Pages
- Doctor Management
- Profile Management

### Node.js

Node.js is a JavaScript runtime environment used for building scalable backend applications.

#### Advantages

- Non-blocking Architecture
- Event-driven Programming
- High Performance
- Fast API Development

#### Usage

- Backend Server
- REST APIs
- JWT Authentication
- MongoDB Connection

## Express.js

Express.js is a lightweight Node.js framework used for creating RESTful web services.

### Features

- Routing
- Middleware Support
- Error Handling
- API Development
- Request Validation

## MongoDB

MongoDB is a NoSQL database used to store application data.

### Advantages

- Flexible Schema
- High Performance
- Scalability
- JSON Document Storage

### Collections

- Users
- Appointments

## Mongoose

Mongoose is an ODM (Object Data Modeling) library for MongoDB.

- Schema Definition
- Data Validation
- CRUD Operations
- Database Queries

## JWT (JSON Web Token)

JWT provides secure authentication between client and server.

- User Authentication
- Authorization
- Protected Routes
- Session Management

## **bcrypt**

bcrypt is used for password encryption.

- Password Hashing
- Secure Storage
- Password Verification

## **Axios**

Axios is an HTTP client used in React applications.

- API Communication
- GET Requests
- POST Requests
- PUT Requests
- DELETE Requests

## **React Router DOM**

Used for navigation between pages.

- Home
- Login
- Register
- Admin Dashboard
- Doctor Dashboard
- Patient Dashboard
- Doctors
- Patients
- Appointments
- Profile

## **CSS**

CSS provides responsive styling and enhances user experience.

- Responsive Layout
- Dashboard Cards
- Navigation Bar
- Sidebar
- Forms
- Tables
- Buttons
- Modern User Interface

## CHAPTER 8

# SYSTEM ARCHITECTURE

The Book Doctor Management System follows a three-tier architecture, separating the application into Presentation Layer, Application Layer, and Data Layer.

### Presentation Layer

The frontend is developed using React.js and consists of reusable components and role-based dashboards.

- Home Page
- Login
- Register
- Admin Dashboard
- Doctor Dashboard
- Patient Dashboard
- Profile
- Appointment Management

### Application Layer

The backend is developed using Node.js and Express.js following the MVC architecture.

- Authentication
- Authorization
- API Routing
- Business Logic
- CRUD Operations
- Appointment Workflow

### Data Layer

MongoDB stores all application data using Mongoose models.

- Users
- Appointments

## System Architecture Flow

|                            |
|----------------------------|
| <b>React Frontend</b>      |
| <b>React Router</b>        |
| <b>Axios HTTP Requests</b> |
| <b>Express.js Server</b>   |
| <b>Routes</b>              |
| <b>Controllers</b>         |
| <b>Services</b>            |
| <b>Mongoose Models</b>     |
| <b>MongoDB Database</b>    |

## CHAPTER 9

# TECHNICAL ARCHITECTURE

The technical architecture integrates frontend, backend, authentication, and database components into a secure web application.

### Frontend

- React.js
- React Router
- Axios
- CSS
- React Icons

### Backend

- Node.js
- Express.js
- MVC Pattern
- JWT Authentication
- bcrypt Encryption

### Database

- MongoDB
- Mongoose

### Security

- JWT Token Verification
- Password Hashing
- Protected Routes
- Role-Based Authorization

### Development Tools

- VS Code
- MongoDB Compass
- Postman
- Git

- GitHub
- npm

# CHAPTER 10

## DATABASE DESIGN

### 10.1 Introduction

The Book Doctor Management System uses MongoDB, a NoSQL document-oriented database, for storing application data. MongoDB stores data in collections using JSON-like BSON documents, providing flexibility and scalability. Mongoose is used as the Object Data Modeling (ODM) library to define schemas, validate data, and perform CRUD operations.

The database is designed to support three different user roles—Admin, Doctor, and Patient—along with appointment management.

### 10.2 Database Collections

The project consists of the following primary collections:

| Collection   | Description                               |
|--------------|-------------------------------------------|
| Users        | Stores Admin, Doctor, and Patient details |
| Appointments | Stores appointment booking details        |

### 10.3 User Collection

The Users collection stores all user information, including Admins, Doctors, and Patients.

| Field     | Data Type | Description        |
|-----------|-----------|--------------------|
| _id       | ObjectId  | Unique Identifier  |
| firstName | String    | User First Name    |
| lastName  | String    | User Last Name     |
| email     | String    | Email Address      |
| password  | String    | Encrypted Password |
| phone     | String    | Mobile Number      |

| Field           | Data Type | Description              |
|-----------------|-----------|--------------------------|
| role            | String    | Admin / Doctor / Patient |
| specialization  | String    | Doctor Specialty         |
| qualification   | String    | Doctor Qualification     |
| experience      | Number    | Years of Experience      |
| consultationFee | Number    | Consultation Charges     |
| hospital        | String    | Hospital Name            |
| address         | String    | Address                  |
| about           | String    | Doctor Description       |
| available       | Boolean   | Availability Status      |
| createdAt       | Date      | Registration Date        |

## 10.4 Appointment Collection

The Appointments collection stores appointment information.

| Field           | Data Type | Description                                 |
|-----------------|-----------|---------------------------------------------|
| _id             | ObjectId  | Appointment ID                              |
| patient         | ObjectId  | Patient Reference                           |
| doctor          | ObjectId  | Doctor Reference                            |
| appointmentDate | Date      | Appointment Date                            |
| appointmentTime | String    | Appointment Time                            |
| reason          | String    | Medical Reason                              |
| status          | String    | Pending / Confirmed / Completed / Cancelled |
| createdAt       | Date      | Booking Date                                |

## 10.5 Database Relationships

The Book Doctor Management System establishes relationships between users and appointments.

**One Doctor → Many Appointments**

One doctor can receive multiple appointment requests.

**One Patient → Many Appointments**

One patient can book multiple appointments.

**One Appointment → One Doctor**

Each appointment belongs to exactly one doctor.

**One Appointment → One Patient**

Each appointment belongs to exactly one patient.

# CHAPTER 11

## FRONTEND DEVELOPMENT

### 11.1 Overview

The frontend of the Book Doctor Management System is developed using React.js with Vite. It provides an interactive, responsive, and user-friendly interface for all system users.

The frontend communicates with the backend through REST APIs using Axios.

### 11.2 Frontend Folder Structure

```
client/  
■  
■■■ public/  
■  
■■■ src/  
■ ■■■ assets/  
■ ■■■ components/  
■ ■■■ context/  
■ ■■■ pages/  
■ ■■■ services/  
■ ■■■ styles/  
■ ■■■ App.jsx  
■ ■■■ main.jsx  
■ ■■■ index.css  
■  
■■■ package.json  
■■■ vite.config.js
```

### 11.3 Major Components

#### Home Page

Displays the landing page introducing the system.

- Navigation
- Login Button
- Register Button

- About Information

## Login Module

Provides secure authentication.

- Email Validation
- Password Validation
- JWT Token Generation
- Role Identification

## Registration Module

Allows new users to register.

- First Name
- Last Name
- Email
- Password
- Phone Number
- Role Selection

## Admin Dashboard

The administrator dashboard manages the complete system.

- View Statistics
- Manage Doctors
- Manage Patients
- Manage Appointments
- Profile

## Doctor Dashboard

Provides doctors with appointment management.

- View Appointments
- Update Appointment Status
- Profile Management

## Patient Dashboard

Provides patients with booking functionality.

- View Doctors

- Book Appointment
- Appointment History
- Profile

## Profile Page

Allows users to update profile information.

- Update Name
- Update Phone Number
- View Email
- View Role

## 11.4 Axios API Integration

Axios is used to connect the frontend with backend REST APIs.

- Authentication APIs
- Doctor APIs
- Appointment APIs
- Profile APIs
- Admin APIs

## 11.5 React Router

The application uses React Router DOM for page navigation.

- /
- /login
- /register
- /admin
- /doctor
- /patient
- /profile
- /patient/doctors
- /patient/book/:id
- /patient/appointments
- /doctor/appointments

## 11.6 Frontend Features

- Responsive Design
- Reusable Components
- Secure Authentication
- Dashboard Navigation
- REST API Integration
- State Management
- Error Handling
- Loading Indicators

## CHAPTER 12

# BACKEND DEVELOPMENT

### 12.1 Overview

The backend is developed using Node.js and Express.js following the MVC (Model View Controller) architecture.

The backend manages authentication, business logic, database operations, and secure communication with the frontend.

### 12.2 Backend Folder Structure

```
server/  
■  
■■■ src/  
■  
■■■ config/  
■  
■■■ controllers/  
■  
■■■ middleware/  
■  
■■■ models/  
■  
■■■ routes/  
■  
■■■ services/  
■  
■■■ app.js  
■  
■■■ server.js
```

### 12.3 Controllers

Controllers receive HTTP requests and return appropriate responses.

- Auth Controller
- Doctor Controller
- Appointment Controller

- Admin Controller
- Profile Controller

## 12.4 Services

The Service Layer contains business logic.

- Authentication
- Doctor Management
- Appointment Management
- Admin Dashboard
- Profile Management

## 12.5 Models

### User Model

- Admin
- Doctor
- Patient

### Appointment Model

- Patient
- Doctor
- Date
- Time
- Status

## 12.6 Routes — Major REST APIs

### Authentication

- POST /api/auth/register
- POST /api/auth/login

### Profile

- GET /api/profile
- PUT /api/profile

### Doctor

- GET /api/doctors

- PUT /api/doctors/update

## Appointment

- POST /api/appointments
- GET /api/appointments
- PUT /api/appointments/:id/status

## Admin

- GET /api/admin/dashboard
- GET /api/admin/users
- GET /api/admin/appointments

## 12.7 Authentication

JWT authentication is implemented to secure protected APIs.

```
User Login
↓
JWT Generated
↓
Token Stored
↓
Frontend Sends Token
↓
Backend Verifies Token
↓
Access Granted
```

## 12.8 Password Security

Passwords are encrypted using bcrypt before storing them in MongoDB.

- Secure Storage
- One-way Hashing
- Password Verification

## 12.9 Middleware

- Authentication Middleware
- Admin Authorization Middleware
- Helmet
- Morgan

- CORS

## **12.10 Backend Features**

- MVC Architecture
- RESTful APIs
- JWT Authentication
- Password Hashing
- MongoDB Integration
- Error Handling
- Role-Based Authorization
- CRUD Operations
- Appointment Status Management
- Secure Database Access

## CHAPTER 13

# API DOCUMENTATION

The backend exposes secure RESTful APIs developed using Express.js. These APIs enable communication between the React frontend and MongoDB database.

### Authentication APIs

| Method | Endpoint           | Description                        |
|--------|--------------------|------------------------------------|
| POST   | /api/auth/register | Register a new user                |
| POST   | /api/auth/login    | Authenticate user and generate JWT |

### Profile APIs

| Method | Endpoint     | Description                |
|--------|--------------|----------------------------|
| GET    | /api/profile | Get logged-in user profile |
| PUT    | /api/profile | Update profile information |

### Doctor APIs

| Method | Endpoint            | Description           |
|--------|---------------------|-----------------------|
| GET    | /api/doctors        | Retrieve all doctors  |
| PUT    | /api/doctors/update | Update doctor profile |

### Appointment APIs

| Method | Endpoint                     | Description               |
|--------|------------------------------|---------------------------|
| POST   | /api/appointments            | Book appointment          |
| GET    | /api/appointments            | View appointments         |
| PUT    | /api/appointments/:id/status | Update appointment status |

### Admin APIs

| Method | Endpoint                | Description           |
|--------|-------------------------|-----------------------|
| GET    | /api/admin/dashboard    | Dashboard statistics  |
| GET    | /api/admin/users        | View all users        |
| GET    | /api/admin/appointments | View all appointments |

## CHAPTER 14

# PROJECT EXECUTION

### Software Requirements

| Software        | Version    |
|-----------------|------------|
| Node.js         | Latest LTS |
| React.js        | 18+        |
| Express.js      | Latest     |
| MongoDB         | 7.x        |
| MongoDB Compass | Latest     |
| VS Code         | Latest     |
| Git             | Latest     |
| GitHub          | Latest     |

### Hardware Requirements

| Component        | Requirement                     |
|------------------|---------------------------------|
| Processor        | Intel Core i3 or above          |
| RAM              | Minimum 4 GB (8 GB Recommended) |
| Storage          | 10 GB Free Space                |
| Operating System | Windows 10/11                   |

### Installation Steps

#### Step 1 — Clone the repository

```
git clone https://github.com/GOWD-HUB/book-a-doctor.git
```

#### Step 2 — Install frontend dependencies

```
cd client
npm install
```

### Step 3 — Install backend dependencies

```
cd server
npm install
```

### Step 4 — Configure Environment Variables

Create a .env file inside the server folder.

```
PORT=5000
MONGO_URI=mongodb://127.0.0.1:27017/book_doctor_db
JWT_SECRET=your_secret_key
```

### Step 5 — Run Backend

```
npm run dev
```

### Step 6 — Run Frontend

```
npm run dev
```

### Step 7 — Open Browser

```
http://localhost:5173
```

### Step 8 — Use the Application

- Register
- Login
- Book Appointment
- Manage Appointments

# TESTING

The application has been tested using Postman for backend APIs and modern web browsers for frontend functionality.

## Functional Testing

| Test Case           | Result |
|---------------------|--------|
| User Registration   | Pass   |
| Login               | Pass   |
| JWT Authentication  | Pass   |
| Doctor List         | Pass   |
| Book Appointment    | Pass   |
| Appointment History | Pass   |
| Doctor Dashboard    | Pass   |
| Admin Dashboard     | Pass   |
| Profile Update      | Pass   |

## Database Testing

MongoDB Compass was used to verify:

- Users Collection
- Appointment Collection
- CRUD Operations

## API Testing

API endpoints were tested using Postman. Verified:

- Status Codes
- Request Validation
- Response Data

- Authentication

## **Browser Testing**

Successfully tested on:

- Google Chrome
- Microsoft Edge

# PROJECT RESULTS

The Book Doctor Management System was successfully developed and tested.

The application fulfills all the intended objectives by providing a secure, scalable, and responsive healthcare appointment management platform.

### Achievements

- ✓ Secure User Authentication
- ✓ Role-Based Dashboards
- ✓ Doctor Management
- ✓ Patient Management
- ✓ Appointment Booking
- ✓ Appointment Status Updates
- ✓ Profile Management
- ✓ JWT Security
- ✓ MongoDB Integration
- ✓ REST API Communication
- ✓ Responsive User Interface

### Demo Screens

The application consists of the following screens:

- Home Page
- Login Page
- Registration Page
- Admin Dashboard
- Doctor Dashboard
- Patient Dashboard
- Doctors Page
- Patients Page
- Appointment Page
- Profile Page

# FUTURE ENHANCEMENTS

The application can be extended with several advanced features.

### Proposed Enhancements

- Online Payment Gateway
- Video Consultation
- AI Healthcare Chatbot
- Email Notifications
- SMS Notifications
- Medical Report Upload
- E-Prescription
- Medicine Reminder
- Doctor Ratings & Reviews
- Appointment Calendar Integration
- Mobile Application
- Multi-language Support
- Hospital Management Integration

# CONCLUSION

The Book Doctor Management System successfully demonstrates the implementation of a full-stack healthcare appointment management platform using the MERN technology stack.

The project integrates React.js, Node.js, Express.js, MongoDB, and Mongoose to provide secure authentication, efficient appointment scheduling, and role-based access control for administrators, doctors, and patients.

The use of JWT authentication, bcrypt password encryption, and the MVC architecture enhances security, maintainability, and scalability.

The responsive frontend, combined with RESTful APIs and MongoDB database integration, provides a complete solution for online doctor appointment management.

The project demonstrates practical knowledge of modern web technologies and software engineering principles, making it suitable for real-world healthcare applications.



# REFERENCES

## CHAPTER

| Resource                 | Link                                                                      |
|--------------------------|---------------------------------------------------------------------------|
| React.js Documentation   | <a href="https://react.dev">https://react.dev</a>                         |
| Node.js Documentation    | <a href="https://nodejs.org">https://nodejs.org</a>                       |
| Express.js Documentation | <a href="https://expressjs.com">https://expressjs.com</a>                 |
| MongoDB Documentation    | <a href="https://www.mongodb.com/docs">https://www.mongodb.com/docs</a>   |
| Mongoose Documentation   | <a href="https://mongoosejs.com">https://mongoosejs.com</a>               |
| JWT Documentation        | <a href="https://jwt.io">https://jwt.io</a>                               |
| Axios Documentation      | <a href="https://axios-http.com">https://axios-http.com</a>               |
| MDN Web Docs             | <a href="https://developer.mozilla.org">https://developer.mozilla.org</a> |

## GITHUB REPOSITORY

| Project Name      | Book Doctor Management System                                                                     |
|-------------------|---------------------------------------------------------------------------------------------------|
| GitHub Repository | <a href="https://github.com/GOWD-HUB/book-a-doctor">https://github.com/GOWD-HUB/book-a-doctor</a> |



### Frontend

- React.js
- React Router
- Axios
- CSS
- React Icons

### Backend

- Node.js
- Express.js
- MongoDB
- Mongoose
- JWT
- bcrypt

### Database

- MongoDB Compass
- Users Collection
- Appointments Collection

### Tools Used

- Visual Studio Code
- Git
- GitHub
- Postman
- MongoDB Compass
- npm

## DOCUMENT SUMMARY

| Item             | Details                       |
|------------------|-------------------------------|
| Project Title    | Book Doctor Management System |
| Technology Stack | MERN Stack                    |
| Frontend         | React.js                      |
| Backend          | Node.js + Express.js          |
| Database         | MongoDB                       |
| Authentication   | JWT + bcrypt                  |
| Architecture     | MVC                           |
| IDE              | VS Code                       |
| API Testing      | Postman                       |
| Version Control  | Git & GitHub                  |